

TP02 - Procesos ETL

Gustavo An Contardi, legajo 182818 Bases de Datos Masivas (11088), 2do cuatrimestre 2026

Las consignas

A partir del dataset Education Data del World Bank Group hay que generar tres archivos CSV.

1. Países. Con `id` (identificador numérico entero autoincremental), `nombre_pais` (nombre oficial del país o entidad regional) y `codigo_pais` (código estándar de 3 letras, ISO Alpha-3). Se pide limpiar registros nulos o faltantes en los campos base y eliminar duplicados.

2. Preguntas. Con `id` autoincremental y `pregunta`, la descripción corta del indicador. Se pide sacar con una expresión regular todo el contenido que esté entre paréntesis, incluidos los paréntesis, aplicar un trim para eliminar los espacios que sobran, y consolidar los indicadores únicos sin duplicados.

3. Educación primaria en años. Con país, código de país y duración de la educación primaria en años. Se pide filtrar el indicador que corresponde, quedarse con la medición de 2023, tratar los valores nulos de ese año especificando en el informe la estrategia adoptada, y ordenar de mayor a menor por duración.

Todo eso hay que resolverlo dos veces, una con pandas en una notebook y otra con Apache Hop en el entorno dockerizado de la materia, y comparar los dos procesos. Las cuatro preguntas conceptuales están respondidas más abajo, cada una con su enunciado.

Sobre el uso de IA

Lo aclaro acá adelante porque cambia cómo se leen mis respuestas.

El código de pandas de la notebook lo generó una IA. Yo decidí qué tenía que hacer cada paso, revisé lo que salía y verifiqué los números, pero no escribí ese código.

Con Apache Hop fue distinto. Los pipelines los armé yo en la interfaz, poniendo cada transform en el lienzo, uniéndolos con hops y llenando cada diálogo a mano. La IA me decía qué transform buscar y qué valor iba en cada campo, pero el armado, la ejecución y los errores fueron míos. De un lado recibí una solución hecha. Del otro la construí yo con ayuda. Esa diferencia es la que hace que pueda contar cuánto tardé en Hop y no pueda contar lo mismo de pandas.

Qué hice

El archivo de entrada tiene 43.624 filas y 69 columnas, en formato ancho: cuatro columnas base y una columna por cada año entre 1960 y 2023. Son 266 países o entidades por 164 indicadores, con la grilla completa, haya dato o no.

Salida	Filas	Pipeline de Hop
países.csv	266	tp02-A-paises.hpl
preguntas.csv	151	tp02-B-preguntas.hpl
primaria_2023.csv	256	tp02-C-primaria.hpl

Los tres archivos que genera Hop son idénticos a los que genera la notebook. Lo verifiqué con una función que compara por contenido y no por formato, porque las dos herramientas escriben distinto las comillas y los saltos de línea. La última celda de la notebook imprime las tres comparaciones.

Hubo que tomar una decisión que no era obvia para que eso funcionara: **ordenar explícitamente antes de numerar**. El `id` autoincremental sale del orden en que llegan las filas. Si las dos implementaciones no ordenan igual, el mismo país queda con un `id` distinto en cada archivo y la comparación falla aunque los datos estén bien. Lo mismo con el desempate de la tercera salida. Hay 167 países que tienen 6 años de primaria, así que sin un segundo criterio de orden el resultado sale en cualquier orden.

La estrategia para los nulos

De los 266 países del indicador `SE.PRM.DURS`, 255 reportan valor en 2023 y 11 no.

Adopté un criterio único: arrastro el último valor reportado y descarto solamente lo que no tiene ningún dato en los 64 años del archivo. De los 11 sin dato en 2023, uno (Afganistán) tiene 53 años de historia y su último valor es de 2022, así que hereda ese 6. Los otros 10 nunca reportaron nada y quedan afuera. Quedan 256 filas.

Elegí el criterio por ausencia de dato y no por tipo de entidad. Es reproducible y no depende de mi opinión sobre qué cuenta como país. Los 10 descartados quedan listados en `C-descartados.csv`, que lo genera el mismo pipeline.

En pandas eso es un `ffill(axis=1)` sobre las columnas de año. En Hop es un transform `*Coalesce fields*`, que devuelve el primer valor no nulo de una lista de campos: pasándole los años de 2023 hacia atrás, el primero no nulo es el último que reportó.

1. Facilidad de desarrollo y curva de aprendizaje

¿Cuál de los dos enfoques (Pandas vs. Apache Hop) resultó más intuitivo para realizar transformaciones de texto complejas como la limpieza mediante expresiones regulares (Regex)? Justifique evaluando el tiempo de desarrollo y la legibilidad de la solución.

Para la parte de texto, pandas.

El patrón es el mismo en las dos herramientas y es lo único realmente difícil del punto 2. Usé `\s*\([^()]*\)` y no `\s*\(.*\)`, porque el cuantificador codicioso `matchea` desde el primer paréntesis hasta el último y rompe cinco indicadores del tipo `School enrollment`, `primary (gross)`, `gender parity index (GPI)`, donde se lleva puesto el `gender parity index`, que no estaba entre paréntesis.

Lo que cambia es dónde termina ese patrón. En la notebook es una línea, con la regex al lado de la operación, y se entiende sola. En Hop hay que llenar una grilla de nueve columnas y el patrón queda guardado adentro de una celda. Mirando el lienzo se ve un transform que dice "Sacar parentesis" y nada más. Para saber qué hace hay que abrirlo. Eso es peor para la legibilidad, aunque el lienzo sea más lindo de mostrar.

Sobre el tiempo, la curva fue muy marcada:

Salida	Tiempo en Hop
A (países)	60 minutos
B (preguntas)	20 minutos
C (primaria)	20 minutos

Era la primera vez que usaba la herramienta, así que casi toda esa primera hora fue aprender la interfaz y no resolver el problema. Tardé un rato largo en darme cuenta de que un transform se agrega con un clic

en el lienzo y se configura con clic derecho y Edit. Después me quedé trabado en la grilla de campos del *CSV file input*, que en la versión web queda con altura cero y no se puede usar, hasta que cambié a *Text file input*, que hace lo mismo pero reparte el trabajo en pestañas. El editor de condición del *Filter rows* también me costó, porque la parte derecha son dos casillas apiladas y hay que saber que la de abajo es la del valor fijo. Y los hops entre transforms hay que crearlos uno por uno.

Nada de eso volvió a aparecer en el segundo pipeline. La tercera salida, que era la más difícil de las tres desde el punto de vista del problema, me llevó el mismo tiempo que la segunda.

Una cosa que quiero decir aparte

Si la IA no existiera, creo que me habría resultado bastante más fácil aprender Apache Hop que pandas arrancando de cero.

En Hop las opciones están a la vista. El diálogo te muestra un desplegable con los tipos, una lista con los campos que hay disponibles, otra con las funciones que podés usar en una condición. No necesitás saber que algo existe para encontrarlo, porque está ahí y lo ves. La primera vez que abrí el editor de condición no sabía que existía `IS NOT NULL`, y lo encontré leyendo la lista.

Con pandas pasa lo contrario. Para imputar el último valor reportado hay que saber que `ffill` existe, y que tiene un parámetro `axis`, antes de poder buscarlo en la documentación. La documentación te contesta lo que preguntás, no te muestra lo que hay. Sin alguien o algo que te diga "esto se hace con `ffill`", el camino es mucho más largo.

Le veo una contra a Hop igual: cuando no te deja hacer algo, no hay vuelta. La grilla rota del *CSV file input* no se arregla con maña, hay que saber que existe otro transform que sirve. En pandas siempre hay otra forma de escribir lo mismo.

2. Manejo de errores, trazabilidad y depuración

Al momento de inspeccionar los datos intermedios o detectar inconsistencias durante la ejecución, ¿qué diferencias encontró entre el flujo gráfico de Apache Hop (sus transformaciones/steps y preview) y el entorno interactivo de una Notebook de Python (Jupyter/Colab)?

Lo que más me sirvió de Hop es que el conteo de filas por paso viene gratis. Cuando termina una ejecución, la pestaña Metrics muestra cuántas filas leyó y escribió cada transform. En la tercera salida eso se lee de corrido: entran 43.624, quedan 266 al filtrar el indicador, pasan 256 el filtro de nulos y 10 se van por la otra rama. En la notebook esa misma trazabilidad existe, pero la tengo que escribir yo con prints en cada paso, y si me olvido de uno no me entero.

Lo segundo es el manejo de rechazos. Un *Filter rows* tiene dos salidas, así que lo que no pasa el filtro se puede escribir a un archivo en vez de desaparecer. Los 10 países descartados de la salida 3 no son un número en un log, son un CSV que puedo abrir y mostrar.

En contra, tres cosas que me pasaron de verdad:

- El preview no funciona igual en todos los transforms. Sobre un *Filter rows* me devolvió "no preview rows found" aunque el pipeline corría perfecto, porque las filas salen por las ramas con nombre y no por una salida común. Hay que mirar el transform siguiente, o ir a Metrics.
- Los nombres de campo se validan recién en ejecución. Escribí `duracion_primaria_datos` en vez de `duracion_primaria_anios` en el *Sort rows* y Hop me dejó guardar sin decir nada. El error apareció cuando pasó la primera fila.

- No todos los errores rojos son errores del pipeline. Al guardar por primera vez me tiró un `SWTException: Invalid thread access` con veinte líneas de traza, y el archivo estaba guardado igual. En otra corrida, un problema de permisos del entorno se presentó como `preparing pipeline execution failed`, con un stack trace que no tenía nada que ver con lo que yo había armado. Perdí un rato largo buscándome el error a mí.

En la notebook los errores apuntan a la línea y puedo revisar cualquier variable del medio abriendo una celda nueva, sin volver a correr nada. La contra es el estado: las celdas se pueden ejecutar en cualquier orden, así que una variable puede tener adentro algo que no corresponde al código que estoy leyendo. En Hop eso no puede pasar, porque el pipeline arranca siempre desde el archivo y siempre desde el principio.

Para explorar y entender los datos me quedo con la notebook. Para una corrida que después alguien tiene que auditar, los contadores por paso y los archivos de rechazo de Hop son más difíciles de falsear.

3. Escalabilidad y rendimiento en procesamiento masivo

Pensando en un escenario de volúmenes masivos de datos (Big Data) que superen la memoria RAM de un solo equipo, ¿cuáles son las limitaciones de pandas y cómo Apache Hop permite gestionar/escalar flujos ETL mediante procesamiento distribuido o en streaming?

La limitación de pandas no es de tamaño, es de modelo. `read_csv` levanta el archivo entero en memoria y el `DataFrame` vive ahí. Mientras entre, anda rápido. Cuando no entra, no hay un modo degradado, falla y listo. Las salidas son leer por chunks, o pasarse a Dask o Polars, y las tres implican reescribir el código, no cambiar un parámetro.

Hop trabaja por filas. Los transforms de un pipeline arrancan todos juntos y las filas los van atravesando de a una, así que la memoria que se usa depende del tamaño de los buffers entre transforms y no del tamaño del archivo. Leer un CSV, filtrar y escribir otro se puede hacer sobre un archivo más grande que la RAM sin cambiar nada.

La excepción son los transforms que bloquean, los que necesitan ver todo antes de largar la primera fila. El `*Sort rows*` es el caso claro, y me pareció interesante que Hop lo resuelva a la vista: en su diálogo hay un campo `*Sort size (rows in memory)*` y un directorio temporal. Cuando la entrada pasa ese tope, el transform escribe archivos intermedios en disco y después los mezcla. Ordenar no está limitado por la RAM, está limitado por el disco, y cuesta más lento. Al `*Unique rows*` le pasa algo parecido, porque exige la entrada ordenada.

Para escalar más allá de una máquina, Hop permite correr el mismo pipeline sobre Apache Beam cambiando la configuración de ejecución, con Spark, Flink o Dataflow como motor. En la instalación que usé aparecen los transforms de Beam en el catálogo. No es gratis: el conjunto de transforms soportados sobre Beam es más chico que el local, así que no cualquier pipeline se migra sin tocar nada. Pero lo que se mueve sigue siendo el mismo archivo `.hpl` y no un código distinto.

En este TP el volumen no fue problema para ninguna de las dos. El contenedor de Hop está limitado a 1 GB de heap y procesó las 43.624 filas sin despeinarse.

4. Portabilidad, automatización y despliegue en producción

Analice el costo de mantenimiento y orquestación en producción de ambas opciones: ¿Qué ventajas ofrece la dockerización de Apache Hop frente a la ejecución e integración de scripts/notebooks de Python mediante herramientas de orquestación (ej. Apache Airflow, Cron, etc.)?

La ventaja más concreta de dockerizar Hop es que el entorno deja de ser un problema. Un `docker-compose up` levanta la versión 2.19.0 exacta, con sus plugins y su JDK, sin instalar nada en mi máquina. La carpeta del proyecto está montada como volumen, así que los pipelines que edito desde el navegador quedan como archivos del repositorio.

Y son archivos de texto. Un `.hpl` es XML: se versiona, se revisa en un pull request y se ve qué cambió entre dos versiones. Un `.ipynb` es JSON con las salidas adentro, y el diff de una notebook es prácticamente ilegible.

Para automatizar, Hop trae `hop-run`, que ejecuta un pipeline desde la línea de comandos sin interfaz gráfica. Eso es lo que permite meterlo en un cron o en un contenedor que arranca, corre y termina. La interfaz queda como herramienta de diseño y no como requisito para ejecutar. Con notebooks hay que sumar `papermill` o `nbconvert`, y además fijar el entorno de Python, que es justamente lo que Docker resuelve del otro lado.

Hop también tiene workflows (`.hwf`), que encadenan pipelines en secuencia y ramifican según cómo termine cada paso. Si uno falla, el flujo se va por la rama de error en vez de seguir con datos incompletos.

Dicho eso, no me parece que sean herramientas equivalentes. Un workflow de Hop orquesta lo que pasa adentro de Hop. Airflow orquesta un sistema entero: calendarios, reintentos, dependencias entre procesos de herramientas distintas, backfills, alertas. Lo razonable es combinarlos, con Airflow disparando un `hop-run` dentro de un contenedor. Y ahí la dockerización vuelve a ser la ventaja, porque el contenedor es la unidad que Airflow sabe manejar.

Limitaciones asumidas

- El dataset incluye agregados regionales (`world`, `Euro area`, `High income`) además de países. Los dejé adentro de las tres salidas, porque la consigna pide "país o entidad regional". `world` aparece en el ranking de primaria con 6 años.
- Al sacar el contenido entre paréntesis, 26 indicadores distintos terminan con el mismo nombre, y quedan 13. Por ejemplo `School enrollment, primary (% gross)` y `(% net)` quedan los dos como `School enrollment, primary`. Los consolidé porque la consigna pide indicadores únicos, pero eso significa que `preguntas.csv` pierde la distinción entre indicadores que miden cosas distintas.